Name of Student : Raghav Mahesh Sarda	**PRN:** 25070122***235***
Subject: Programming with Java	**Semester: III**
ASSIGNMENT No : 20	**Year: AY 2026-27**
TITLE : JDBC INSERT, UPDATE, and DELETE Operations	

Problem Statement

Develop a Java application to establish connectivity with a database and perform INSERT, UPDATE, and DELETE operations.

Learning Objectives

To perform database modification operations using JDBC.

Program:

sit_java.sql

```
CREATE DATABASE sit_java;
USE sit_java;

-- Assignment 19
-- Table for Exercise 1
CREATE TABLE student (
   roll_no INT PRIMARY KEY,
   name VARCHAR(50),
   course VARCHAR(50),
   marks DOUBLE
);

INSERT INTO student VALUES
(101, 'Raghav Sarda', 'Computer Science', 92.5),
(102, 'Parth Kulkarni Sarda', 'Computer Science', 94.5),
(103, 'Yatharth Shah', 'Computer Science', 93.5);

-- Table for Exercise 2
CREATE TABLE product (
   product_id INT PRIMARY KEY,
   product_name VARCHAR(50),
   quantity INT,
```

```
   price DOUBLE
);

INSERT INTO product VALUES
(501, 'Logitech MX Master 4 Mouse', 10, 16000),
(502, 'Apple Magic Keyboard', 20, 18000),
(503, 'Apple Studio Display', 15, 179999);


-- Assignment 20
-- Table for Exercise 1
CREATE TABLE employee (
   emp_id INT PRIMARY KEY,
   name VARCHAR(50),
   department VARCHAR(50),
   salary DOUBLE
);

-- Table for Exercise 2
CREATE TABLE student_crud (
   roll_no INT PRIMARY KEY,
   name VARCHAR(50),
   course VARCHAR(50),
   marks DOUBLE
);
```

EmployeeCRUD.java

```
// Assignment 20 - Exercise 1: Employee Records CRUD Operations
import java.sql.*;
import java.util.*;
public class EmployeeCRUD
{
   public static void main(String args[]) // Main Method
   {
       Scanner sc = new Scanner(System.in);
       String url = "jdbc:mysql://localhost:3306/sit_java"; // DB URL
       String user = "thegharkalaptop"; // DB Username
       String pass = ""; // DB Password

       try
       {
           Class.forName("com.mysql.cj.jdbc.Driver"); // Registering
driver
```

```
            Connection con = DriverManager.getConnection(url, user,
pass); // Connecting to DB

            while(true) // Menu loop
            {
                System.out.println("\n-- Employee CRUD Operations --");
// Menu
                System.out.println("1. Create (Insert) Employee");
                System.out.println("2. Read (View) Employees");
                System.out.println("3. Update Employee Salary");
                System.out.println("4. Delete Employee Record");
                System.out.println("5. Exit");
                System.out.print("Enter choice: ");
                int choice = sc.nextInt();

                if(choice == 1) // INSERT
                {
                    System.out.print("Enter ID: ");
                    int id = sc.nextInt();
                    sc.nextLine(); // Consuming newline
                    System.out.print("Enter Name: ");
                    String name = sc.nextLine();
                    System.out.print("Enter Department: ");
                    String dept = sc.nextLine();
                    System.out.print("Enter Salary: ");
                    double sal = sc.nextDouble();

                    String sql = "INSERT INTO employee VALUES (?, ?, ?,
?)"; // Parameterized query
                    PreparedStatement ps = con.prepareStatement(sql); //
Preparing statement
                    ps.setInt(1, id);
                    ps.setString(2, name);
                    ps.setString(3, dept);
                    ps.setDouble(4, sal);
                    ps.executeUpdate(); // Executing DML
                    System.out.println("Employee record inserted
successfully!");
                    ps.close(); // Closing PreparedStatement
                }
                else if(choice == 2) // SELECT
                {
                    Statement stmt = con.createStatement(); // Creating
Statement
```

```
                    ResultSet rs = stmt.executeQuery("SELECT * FROM
employee"); // Executing SELECT

System.out.println("\nID\tName\t\tDepartment\tSalary");
                    while(rs.next()) // Traversing records
                    {
                        System.out.println(rs.getInt(1) + "\t" +
rs.getString(2) + "\t" + rs.getString(3) + "\t" + rs.getDouble(4));
                    }
                    rs.close();
                    stmt.close(); // Closing Statement
                }
                else if(choice == 3) // UPDATE
                {
                    System.out.print("Enter Employee ID to update: ");
                    int id = sc.nextInt();
                    System.out.print("Enter New Salary: ");
                    double newSal = sc.nextDouble();

                    String sql = "UPDATE employee SET salary = ? WHERE
emp_id = ?"; // UPDATE query
                    PreparedStatement ps = con.prepareStatement(sql); //
Preparing statement
                    ps.setDouble(1, newSal);
                    ps.setInt(2, id);
                    int rows = ps.executeUpdate(); // Executing update
                    if(rows > 0) System.out.println("Salary updated
successfully!");
                    else System.out.println("Employee ID not found.");
                    ps.close(); // Closing PreparedStatement
                }
                else if(choice == 4) // DELETE
                {
                    System.out.print("Enter Employee ID to delete: ");
                    int id = sc.nextInt();

                    String sql = "DELETE FROM employee WHERE emp_id = ?";
// DELETE query
                    PreparedStatement ps = con.prepareStatement(sql); //
Preparing statement
                    ps.setInt(1, id); // Binding ID
                    int rows = ps.executeUpdate(); // Executing delete
                    if(rows > 0) System.out.println("Employee deleted
successfully!");
```

```
                    else System.out.println("Employee ID not found.");
                    ps.close(); // Closing PreparedStatement
                }
                else if(choice == 5) // EXIT
                {
                    System.out.println("Exiting application...");
                    break;
                }
            }
            con.close(); // Closing DB Connection
        }
        catch(Exception e) // Catching exceptions
        {
            System.out.println("Error: " + e.getMessage());
        }
        sc.close();
    }
}
```

StudentCRUD.java

```
// Assignment 20 - Exercise 2: Student Records CRUD Operations (Roll,
Name, Course, Marks)
import java.sql.*;
import java.util.*;
public class StudentCRUD
{
    public static void main(String args[]) // Main Method
    {
        Scanner sc = new Scanner(System.in);
        String url = "jdbc:mysql://localhost:3306/sit_java"; // DB URL
        String user = "thegharkalaptop"; // DB Username
        String pass = ""; // DB Password

        try // Try block
        {
            Class.forName("com.mysql.cj.jdbc.Driver"); // Registering
driver
            Connection con = DriverManager.getConnection(url, user,
pass); // Connecting to DB

            while(true)
            {
```

```
                System.out.println("\n-- Student Management System (CRUD)
--"); // Menu
                System.out.println("1. Add Student (Create)");
                System.out.println("2. View All Students (Read)");
                System.out.println("3. Update Student Marks (Update)");
                System.out.println("4. Delete Student Record (Delete)");
                System.out.println("5. Exit");
                System.out.print("Enter choice: ");
                int choice = sc.nextInt();

                if(choice == 1) // CREATE
                {
                    System.out.print("Enter Roll Number: ");
                    int roll = sc.nextInt();
                    sc.nextLine(); // Consuming newline
                    System.out.print("Enter Student Name: ");
                    String name = sc.nextLine();
                    System.out.print("Enter Course: ");
                    String course = sc.nextLine();
                    System.out.print("Enter Marks: ");
                    double marks = sc.nextDouble();

                    String sql = "INSERT INTO student_crud VALUES (?, ?,
?, ?)"; // Parameterized query
                    PreparedStatement ps = con.prepareStatement(sql); //
Preparing query
                    ps.setInt(1, roll);
                    ps.setString(2, name);
                    ps.setString(3, course);
                    ps.setDouble(4, marks);
                    ps.executeUpdate(); // Executing insert
                    System.out.println("Student record added
successfully!");
                    ps.close(); // Closing statement
                }
                else if(choice == 2) // READ
                {
                    Statement stmt = con.createStatement(); // Creating
statement
                    ResultSet rs = stmt.executeQuery("SELECT * FROM
student_crud"); // Executing read
                    System.out.println("\nRoll
No\tName\t\tCourse\t\tMarks");
                    while(rs.next()) // Looping through results
```

```
                    {
                        System.out.println(rs.getInt(1) + "\t" +
rs.getString(2) + "\t" + rs.getString(3) + "\t" + rs.getDouble(4));
                    }
                    rs.close(); // Closing ResultSet
                    stmt.close(); // Closing Statement
                }
                else if(choice == 3) // UPDATE
                {
                    System.out.print("Enter Roll Number to update: ");
                    int roll = sc.nextInt();
                    System.out.print("Enter New Marks: ");
                    double newMarks = sc.nextDouble();

                    String sql = "UPDATE student_crud SET marks = ? WHERE
roll_no = ?"; // Update query
                    PreparedStatement ps = con.prepareStatement(sql); //
Preparing statement
                    ps.setDouble(1, newMarks);
                    ps.setInt(2, roll);
                    int rows = ps.executeUpdate(); // Executing update
                    if(rows > 0) System.out.println("Marks updated
successfully!");
                    else System.out.println("Student roll number not
found.");
                    ps.close(); // Closing statement
                }
                else if(choice == 4) // DELETE
                {
                    System.out.print("Enter Roll Number to delete: ");
                    int roll = sc.nextInt();

                    String sql = "DELETE FROM student_crud WHERE roll_no
= ?"; // Delete query
                    PreparedStatement ps = con.prepareStatement(sql); //
Preparing statement
                    ps.setInt(1, roll);
                    int rows = ps.executeUpdate(); // Executing deletion
                    if(rows > 0) System.out.println("Student record
deleted successfully!");
                    else System.out.println("Student roll number not
found.");
                    ps.close(); // Closing statement
                }
```

```
                else if(choice == 5) // EXIT
                {
                    System.out.println("Exiting Student System...");
                    break;
                }
            }
            con.close(); // Closing connection
        }
        catch(Exception e) // Catching exceptions
        {
            System.out.println("Error: " + e.getMessage());
        }
        sc.close();
    }
}
```

Output:

EmployeeCRUD.java

```
Problems  Output  Debug Console  Terminal  Ports

thegharkalaptop@The-Ghar-Ka-Laptop JavaSQL % javac EmployeeCRUD.java && java -cp ".:lib/*" EmployeeCRUD

-- Employee CRUD Operations --
1. Create (Insert) Employee
2. Read (View) Employees
3. Update Employee Salary
4. Delete Employee Record
5. Exit
Enter choice: 1
Enter ID: 235
Enter Name: Raghav Sarda
Enter Department: CSE & Ex-AI & ML
Enter Salary: 123456789
Employee record inserted successfully!

-- Employee CRUD Operations --
1. Create (Insert) Employee
2. Read (View) Employees
3. Update Employee Salary
4. Delete Employee Record
5. Exit
Enter choice: 2

ID      Name            Department      Salary
235     Raghav Sarda    CSE & Ex-AI & ML        1.23456789E8

-- Employee CRUD Operations --
1. Create (Insert) Employee
2. Read (View) Employees
3. Update Employee Salary
4. Delete Employee Record
5. Exit
Enter choice: 3
Enter Employee ID to update: 235
Enter New Salary: 1234567890
Salary updated successfully!

-- Employee CRUD Operations --
1. Create (Insert) Employee
2. Read (View) Employees
3. Update Employee Salary
4. Delete Employee Record
5. Exit
Enter choice: 2

ID      Name            Department      Salary
235     Raghav Sarda    CSE & Ex-AI & ML        1.23456789E9

-- Employee CRUD Operations --
1. Create (Insert) Employee
2. Read (View) Employees
3. Update Employee Salary
4. Delete Employee Record
5. Exit
Enter choice: 4
Enter Employee ID to delete: 235
Employee deleted successfully!

-- Employee CRUD Operations --
1. Create (Insert) Employee
2. Read (View) Employees
3. Update Employee Salary
4. Delete Employee Record
5. Exit
Enter choice: 2

ID      Name            Department      Salary

-- Employee CRUD Operations --
1. Create (Insert) Employee
2. Read (View) Employees
3. Update Employee Salary
4. Delete Employee Record
5. Exit
Enter choice: 5
Exiting application...
thegharkalaptop@The-Ghar-Ka-Laptop JavaSQL %
```

StudentCRUD.java

```
Problems  Output  Debug Console  Terminal  Ports

thegharkalaptop@The-Ghar-Ka-Laptop JavaSQL % javac StudentCRUD.java && java -cp ".:lib/*" StudentCRUD

 -- Student Management System (CRUD) --
 1. Add Student (Create)
 2. View All Students (Read)
 3. Update Student Marks (Update)
 4. Delete Student Record (Delete)
 5. Exit
 Enter choice: 1
 Enter Roll Number: 235
 Enter Student Name: Raghav Sarda
 Enter Course: JAVA
 Enter Marks: 90
 Student record added successfully!

 -- Student Management System (CRUD) --
 1. Add Student (Create)
 2. View All Students (Read)
 3. Update Student Marks (Update)
 4. Delete Student Record (Delete)
 5. Exit
 Enter choice: 2

 Roll No Name            Course          Marks
 235     Raghav Sarda    JAVA    90.0

 -- Student Management System (CRUD) --
 1. Add Student (Create)
 2. View All Students (Read)
 3. Update Student Marks (Update)
 4. Delete Student Record (Delete)
 5. Exit
 Enter choice: 3
 Enter Roll Number to update: 235
 Enter New Marks: 97
 Marks updated successfully!

 -- Student Management System (CRUD) --
 1. Add Student (Create)
 2. View All Students (Read)
 3. Update Student Marks (Update)
 4. Delete Student Record (Delete)
 5. Exit
 Enter choice: 2

 Roll No Name            Course          Marks
 235     Raghav Sarda    JAVA    97.0

 -- Student Management System (CRUD) --
 1. Add Student (Create)
 2. View All Students (Read)
 3. Update Student Marks (Update)
 4. Delete Student Record (Delete)
 5. Exit
 Enter choice: 4
 Enter Roll Number to delete: 235
 Student record deleted successfully!

 -- Student Management System (CRUD) --
 1. Add Student (Create)
 2. View All Students (Read)
 3. Update Student Marks (Update)
 4. Delete Student Record (Delete)
 5. Exit
 Enter choice: 2

 Roll No Name            Course          Marks

 -- Student Management System (CRUD) --
 1. Add Student (Create)
 2. View All Students (Read)
 3. Update Student Marks (Update)
 4. Delete Student Record (Delete)
 5. Exit
 Enter choice: 5
 Exiting Student System...
thegharkalaptop@The-Ghar-Ka-Laptop JavaSQL %
```

Conclusion:

In this assignment, relational database modification operations comprising INSERT, UPDATE, and DELETE were successfully implemented and verified through JDBC. Using PreparedStatement enabled parameterized query execution, preventing SQL injection vulnerabilities while managing entities for employee and student records. The continuous menu-driven operations demonstrated transactional integrity and accurate synchronization between user input and database storage.